Perl Code Diplomarbeit

Ich schreibe eine Diplomarbeit über Smart Home und Smart Home Systeme. Mein Kollege 1 macht ein Modellhaus und verkabelt alles, mein anderer Kollege 2 macht ein Smart Home System zu diesem Haus mit Home Assist und ich muss nun auch das Smart Home mit dem Smart Home System FHEM verbinden. Wir benutzen NODERED, MQTT, einen Raspi, einen Arduino und ich schreibe meine Codes in VSCODE. Wir kann ich diese Diplomarbeit also meinen Praktischen teil umsetzen und darüber schreiben.

Sehr gut 👍 das ist eine **sehr klare Aufgabenverteilung** – und genau daraus kannst du einen starken praktischen Teil für deine Diplomarbeit machen.

Du bist also zuständig für:

- Smart Home System: **FHEM**
- Verbindung zu:
  - **Node-RED**
  - **MQTT**
  - **Raspberry Pi**
  - **Arduino**
  - paralleles System: **Home Assistant**

Das ist perfekt, weil du damit ein eigenes, klar abgegrenztes System entwickelst.

🔧 1️⃣ Deine Rolle im Projekt (so kannst du es in der Arbeit formulieren)

Dein praktischer Teil lautet im Kern:

> „Implementierung eines Smart-Home-Systems mit FHEM und Integration in eine MQTT-basierte Systemarchitektur“

Du entwickelst also:

- die **FHEM-Installation**
- die **Geräte-Definitionen**
- die **MQTT-Kommunikation**
- die **Automatisierungen**
- die **Anbindung an Node-RED**
- eventuell Vergleich zu Home Assistant

🧱 2️⃣ Technische Gesamtarchitektur (für deine Diplomarbeit)

So sieht eure Architektur logisch aus:

```
Sensoren → Arduino → MQTT → Raspberry Pi → FHEM
                                  ↓
                              Node-RED
                                  ↓
                          Home Assistant
```

Du kannst dazu ein eigenes Architekturdiagramm zeichnen (sehr wichtig für Diplomarbeit!).

📦 3️⃣ Konkrete Umsetzung – Schritt für Schritt

Jetzt wird's wichtig:
Was genau machst du praktisch?

🔹 Schritt 1: Raspberry Pi vorbereiten

Auf dem Raspberry Pi installierst du:

- FHEM
- Mosquitto (MQTT Broker)
- Node-RED
- ggf. Docker (wenn gewünscht)

In deiner Arbeit beschreibst du:

- Warum Raspberry Pi?
- Warum MQTT?
- Warum FHEM?
- Vorteile gegenüber Cloud-Systemen

🔹 Schritt 2: MQTT als zentrale Schnittstelle

MQTT ist euer Kommunikationsstandard.

Beispiel:

Arduino sendet:

```
home/wohnzimmer/temperatur 23.5
```

FHEM subscribed:

```
home/wohnzimmer/#
```

Node-RED verarbeitet:

- Logik

- Visualisierung
- Weiterleitung

In deiner Diplomarbeit erklärst du:

- Publish / Subscribe Prinzip
- Broker
- Topics
- QoS

🔹 Schritt 3: Arduino als Sensoreinheit

Beispiel:

- DHT22 → Temperatur & Luftfeuchtigkeit
- Relais → Lichtsteuerung

Arduino sendet per MQTT Daten an den Broker.

🔹 Schritt 4: FHEM konfigurieren

In FHEM definierst du:

MQTT Device

```
define mqttBroker MQTT2_CLIENT 127.0.0.1:1883
attr mqttBroker room MQTT
```

Temperatur Sensor

```
define WZ_Temperatur MQTT2_DEVICE
attr WZ_Temperatur readingList home/wohnzimmer/temperatur:.* { json2nameValue($EVENT) }
attr WZ_Temperatur room Wohnzimmer
```

Lichtsteuerung

```
define WZ_Licht MQTT2_DEVICE
attr WZ_Licht setList on off
attr WZ_Licht room Wohnzimmer
```

🔹 Schritt 5: Automatisierung in FHEM

Beispiel:

```
define LichtAutomatik notify WZ_Temperatur:temperature:.* {
  if(ReadingsVal("WZ_Temperatur","temperature",0) > 25) {
    fhem("set WZ_Licht on");
  }
}
```

In der Arbeit erklärst du:

- notify
- readings
- Events
- Perl innerhalb von FHEM

🔹 Schritt 6: Node-RED Integration

In Node-RED kannst du:

- MQTT In Node
- Function Node
- MQTT Out Node
- Dashboard

Beispiel:

- Temperatur anzeigen
- Manuelles Schalten
- Daten weiter an Home Assistant

🔹 Schritt 7: Vergleich mit Home Assistant

Da dein Kollege Home Assistant macht, kannst du:

FHEM	Home Assistant
Perl basiert	YAML basiert
sehr flexibel	benutzerfreundlicher
mehr manuelle Konfiguration	mehr GUI

Das gibt deiner Arbeit Tiefe.

🧠 4️⃣ Wie du deinen praktischen Teil aufbauen kannst

Hier eine perfekte Kapitelstruktur für dich:

---

5 Praktische Umsetzung des Smart-Home-Systems mit FHEM

5.1 Systemarchitektur

- Komponenten
- Kommunikationsstruktur

- Diagramm

5.2 Installation und Konfiguration

- Raspberry Pi Setup
- MQTT Broker
- FHEM Installation

5.3 Implementierung der Sensorik

- Arduino
- MQTT Topics
- Datenstruktur

5.4 Implementierung der Aktorik

- Relais
- Lichtsteuerung
- Heizungssteuerung

5.5 Automatisierung mit FHEM

- Events
- Perl Code
- Logikverarbeitung

5.6 Integration mit Node-RED

- Flows
- Datenvisualisierung

5.7 Vergleich mit Home Assistant

📊 5️⃣ Was deine Diplomarbeit stark macht

Damit bekommst du gute Noten:

✔️ Architekturdiagramm
✔️ Codebeispiele
✔️ Erklärung der Befehle
✔️ Sicherheitsaspekte (z.B. kein offener MQTT Port)
✔️ Reflexion (Was hat gut funktioniert? Was war schwierig?)
✔️ Vergleich FHEM vs Home Assistant

🚀 6️⃣ Wie du es praktisch angehen solltest

Ich würde dir empfehlen:

1. Zuerst MQTT sauber testen (mit MQTT Explorer)
2. Dann Arduino Daten senden lassen
3. Dann FHEM subscriben
4. Dann Node-RED einbauen
5. Dann Automatisierungen
6. Zum Schluss Dokumentation schreiben

💡 7️⃣ Wichtig für dich (18 Jahre, Diplomarbeit!)

Du solltest zeigen, dass du:

- eine neue Programmiersprache (Perl) gelernt hast
- ein verteiltes System verstehst
- Netzwerktechnik verstehst
- Automatisierung erklären kannst

Das ist für dein Alter sehr stark 💪

Wenn du möchtest, kann ich dir als Nächstes:

- ✅ ein komplettes Architekturkonzept formulieren (fertig für die Diplomarbeit)
- ✅ ein vollständiges FHEM-Konfigurationsbeispiel erstellen
- ✅ oder deinen praktischen Teil als fertigen Text formulieren

Was brauchst du als Nächstes?

> Schreibe mir bitte den kompletten perl Code für diese Diplomarbeit, also für Lichtsteuerung, eine Heizung zu steuern bzw. einen gewissen Grad Celcius Wert angeben und immer in der Nähe bleiben und wir bauen die komplette Wetterstation mit ein

Unten ist ein **komplettes, „echtes" Perl-Modul für FHEM** (kein Pseudo-Code), das genau deine drei Bereiche abdeckt:

- **Lichtsteuerung** (ON/OFF + optional PWM/Helligkeit)
- **Heizung regeln** (Zieltemperatur setzen + **Hysterese-Regler**, damit er „in der Nähe bleibt")
- **Wetterstation** (MQTT-Readings sammeln und in FHEM als Readings verfügbar machen)

Das Modul arbeitet so, wie man es in einer Diplomarbeit sauber erklären kann: **MQTT ist die Schnittstelle**, Arduino publisht Sensordaten, FHEM wertet aus und publisht Befehle zurück.

1) Datei: `99_DA_SmartHome.pm` (in FHEM ablegen)

Lege diese Datei auf dem Raspberry Pi in folgendem Pfad ab:

- **`/opt/fhem/FHEM/99_DA_SmartHome.pm`**

Dann FHEM neu starten.

```
##############################################################################
# 99_DA_SmartHome.pm
# Diplomarbeit: FHEM SmartHome Logik + MQTT Anbindung (Licht, Heizung, Wetter)
#
# Voraussetzungen in FHEM:
# - MQTT2_CLIENT (Broker-Verbindung)
# - MQTT2_DEVICE für Sensoren/Aktoren (oder direkte publish/subscribe Topics)
#
# Autor: (dein Name)
# Datum: (Datum)
##############################################################################

package main;

use strict;
use warnings;
use Time::HiRes qw(time);

# FHEM Core Funktionen werden zur Laufzeit bereitgestellt:
# Log3($name,$lvl,$msg), readingsSingleUpdate, readingsBeginUpdate, readingsBulkUpdate, readingsEndUpd
# InternalTimer, RemoveInternalTimer, AttrVal, ReadingsVal, ReadingsNum, CommandSet, CommandAttr, Comm

# ----------------------------
# Initialize
# ----------------------------
sub DA_SmartHome_Initialize($) {
  my ($hash) = @_;

  $hash->{DefFn}    = "DA_SmartHome_Define";
  $hash->{UndefFn}  = "DA_SmartHome_Undef";
  $hash->{SetFn}    = "DA_SmartHome_Set";
  $hash->{NotifyFn} = "DA_SmartHome_Notify";
  $hash->{AttrFn}   = "DA_SmartHome_Attr";

  $hash->{AttrList} =
      "mqttLightTopic "
    . "mqttLightBrightnessTopic "
    . "mqttHeatTopic "
    . "mqttWeatherPrefix "
    . "tempDevice tempReading "
    . "targetTemp hysteresis "
    . "minSwitchInterval "
    . "heatOnPayload heatOffPayload "
    . "lightOnPayload lightOffPayload "
    . "brightnessMin brightnessMax "
    . "disable:0,1 "
    . $readingFnAttributes;

  return;
}

# ----------------------------
# Define
# define <name> DA_SmartHome
# ----------------------------
sub DA_SmartHome_Define($$) {
  my ($hash, $def) = @_;
  my @a = split("[ \t]+", $def);

  return "Usage: define <name> DA_SmartHome" if (@a < 3);

  my $name = $hash->{NAME};

  # Defaults (kannst du über Attrs überschreiben)
  CommandAttr(undef, "$name mqttLightTopic home/wohnzimmer/licht/cmd")            if (!defined AttrVal
  CommandAttr(undef, "$name mqttLightBrightnessTopic home/wohnzimmer/licht/pwm") if (!defined AttrVal
  CommandAttr(undef, "$name mqttHeatTopic home/heizung/cmd")                     if (!defined AttrVal
  CommandAttr(undef, "$name mqttWeatherPrefix home/wetter")                      if (!defined AttrVal

  CommandAttr(undef, "$name tempDevice WZ_Temp")     if (!defined AttrVal($name, "tempDevice", undef)
  CommandAttr(undef, "$name tempReading temperature") if (!defined AttrVal($name, "tempReading", unde

  CommandAttr(undef, "$name targetTemp 21.5")        if (!defined AttrVal($name, "targetTemp", undef)
  CommandAttr(undef, "$name hysteresis 0.4")         if (!defined AttrVal($name, "hysteresis", undef)
  CommandAttr(undef, "$name minSwitchInterval 30")   if (!defined AttrVal($name, "minSwitchInterval",
```

```
  CommandAttr(undef, "$name heatOnPayload {\"state\":\"ON\"}")   if (!defined AttrVal($name, "heatOnPa
  CommandAttr(undef, "$name heatOffPayload {\"state\":\"OFF\"}") if (!defined AttrVal($name, "heatOffP

  CommandAttr(undef, "$name lightOnPayload {\"state\":\"ON\"}")   if (!defined AttrVal($name, "lightOn
  CommandAttr(undef, "$name lightOffPayload {\"state\":\"OFF\"}") if (!defined AttrVal($name, "lightOf

  CommandAttr(undef, "$name brightnessMin 0")        if (!defined AttrVal($name, "brightnessMin", unde
  CommandAttr(undef, "$name brightnessMax 255")      if (!defined AttrVal($name, "brightnessMax", unde

  $hash->{STATE} = "initialized";
  readingsSingleUpdate($hash, "state", "initialized", 1);

  # interne Variablen
  $hash->{helper}{lastHeatSwitchTs}  = 0;
  $hash->{helper}{heatState}         = "unknown";
  $hash->{helper}{lastLightCmdTs}    = 0;

  Log3($name, 3, "DA_SmartHome ($name): defined");

  return;
}

# ----------------------------
# Undef
# ----------------------------
sub DA_SmartHome_Undef($$) {
  my ($hash, $arg) = @_;
  RemoveInternalTimer($hash);
  return;
}

# ----------------------------
# Attr
# ----------------------------
sub DA_SmartHome_Attr($$$$) {
  my ($cmd, $name, $attrName, $attrVal) = @_;
  # hier könnte man Validierungen einbauen
  return;
}

# ----------------------------
# Set
# set <name> light on|off
# set <name> brightness <0..255>
# set <name> targetTemp <float>
# set <name> heat on|off|auto
# ----------------------------
sub DA_SmartHome_Set($@) {
  my ($hash, @a) = @_;
  my $name = $hash->{NAME};

  return "device is disabled" if (AttrVal($name, "disable", "0") eq "1");

  return "Unknown argument, choose one of "
    . "light:on,off "
    . "brightness "
    . "targetTemp "
    . "heat:on,off,auto"
    if (@a < 2);

  my $cmd = $a[1];

  if ($cmd eq "light") {
    return "Usage: set $name light on|off" if (@a < 3);
    my $v = lc($a[2]);
    return "Only on/off allowed" if ($v ne "on" && $v ne "off");

    my $topic = AttrVal($name, "mqttLightTopic", "home/wohnzimmer/licht/cmd");
    my $payload = ($v eq "on")
      ? AttrVal($name, "lightOnPayload", "{\"state\":\"ON\"}")
      : AttrVal($name, "lightOffPayload","{\"state\":\"OFF\"}");

    _publish_mqtt($hash, $topic, $payload);
```

```
    readingsBeginUpdate($hash);
    readingsBulkUpdate($hash, "light", $v);
    readingsBulkUpdate($hash, "state", "light_$v");
    readingsEndUpdate($hash, 1);

    $hash->{helper}{lastLightCmdTs} = time();
    Log3($name, 3, "DA_SmartHome ($name): light -> $v");
    return;
  }

  if ($cmd eq "brightness") {
    return "Usage: set $name brightness <0..255>" if (@a < 3);
    my $b = $a[2];
    return "brightness must be a number" if ($b !~ /^-?\d+(\.\d+)?$/);

    my $min = AttrVal($name, "brightnessMin", 0);
    my $max = AttrVal($name, "brightnessMax", 255);

    $b = int($b);
    $b = $min if ($b < $min);
    $b = $max if ($b > $max);

    my $topic = AttrVal($name, "mqttLightBrightnessTopic", "home/wohnzimmer/licht/pwm");
    my $payload = "{\"brightness\":$b}";

    _publish_mqtt($hash, $topic, $payload);

    readingsBeginUpdate($hash);
    readingsBulkUpdate($hash, "brightness", $b);
    readingsBulkUpdate($hash, "state", "brightness_$b");
    readingsEndUpdate($hash, 1);

    Log3($name, 3, "DA_SmartHome ($name): brightness -> $b");
    return;
  }

  if ($cmd eq "targetTemp") {
    return "Usage: set $name targetTemp <float>" if (@a < 3);
    my $t = $a[2];
    return "targetTemp must be numeric" if ($t !~ /^-?\d+(\.\d+)?$/);

    CommandAttr(undef, "$name targetTemp $t");

    readingsBeginUpdate($hash);
    readingsBulkUpdate($hash, "targetTemp", $t);
    readingsBulkUpdate($hash, "state", "targetTemp_$t");
    readingsEndUpdate($hash, 1);

    Log3($name, 3, "DA_SmartHome ($name): targetTemp -> $t");

    # sofort neu regeln
    _control_heating($hash);

    return;
  }

  if ($cmd eq "heat") {
    return "Usage: set $name heat on|off|auto" if (@a < 3);
    my $mode = lc($a[2]);
    return "Only on/off/auto allowed" if ($mode ne "on" && $mode ne "off" && $mode ne "auto");

    readingsBeginUpdate($hash);
    readingsBulkUpdate($hash, "heatMode", $mode);
    readingsBulkUpdate($hash, "state", "heatMode_$mode");
    readingsEndUpdate($hash, 1);

    if ($mode eq "on") {
      _switch_heating($hash, "on");
    } elsif ($mode eq "off") {
      _switch_heating($hash, "off");
    } else {
      _control_heating($hash);
    }

    Log3($name, 3, "DA_SmartHome ($name): heatMode -> $mode");
    return;
```

```
    return;
  }

  return "Unknown argument, choose one of light brightness targetTemp heat";
}

# ----------------------------
# Notify: reagiert auf Events von Sensoren (Temp + Wetterstation)
# - Temperaturgerät liefert z.B. reading "temperature"
# - Wetterstation publish Topics -> in FHEM als MQTT2_DEVICE Readings oder als eigenes Wetter-Device
# ----------------------------
sub DA_SmartHome_Notify($$) {
  my ($hash, $devhash) = @_;
  my $name = $hash->{NAME};

  return if (AttrVal($name, "disable", "0") eq "1");

  my $devName = $devhash->{NAME};

  my $tempDev     = AttrVal($name, "tempDevice", "WZ_Temp");
  my $tempReading = AttrVal($name, "tempReading", "temperature");

  # 1) Temperatur-Update -> Heizung regeln (wenn auto)
  if ($devName eq $tempDev) {
    my $events = deviceEvents($devhash, 1);
    return if (!$events);

    foreach my $e (@{$events}) {
      # Beispiel Event: "temperature: 23.5"
      if ($e =~ /^\Q$tempReading\E:\s*([-+]?\d+(\.\d+)?)/) {
        my $cur = $1 + 0;

        readingsBeginUpdate($hash);
        readingsBulkUpdate($hash, "currentTemp", $cur);
        readingsBulkUpdate($hash, "state", "temp_update");
        readingsEndUpdate($hash, 1);

        Log3($name, 4, "DA_SmartHome ($name): temp update -> $cur");

        _control_heating($hash);
        last;
      }
    }
  }

  # 2) Wetterstation: wenn du ein MQTT2_DEVICE "Wetterstation" hast,
  # kannst du dessen Readings hier einfach spiegeln/standardisieren.
  my $weatherPrefix = AttrVal($name, "mqttWeatherPrefix", "home/wetter");

  # Wenn dein Wetterdevice z.B. "Wetterstation" heißt, trag es als Attribut ein
  # oder lass es so und prüfe per Name:
  if ($devName =~ /Wetter/i) {
    my $events = deviceEvents($devhash, 1);
    return if (!$events);

    my %map = (
      temperature => "weather_temperature",
      humidity    => "weather_humidity",
      pressure    => "weather_pressure",
      wind        => "weather_wind",
      rain        => "weather_rain",
      lux         => "weather_lux",
    );

    readingsBeginUpdate($hash);
    foreach my $e (@{$events}) {
      # generisch: "<reading>: <value>"
      if ($e =~ /^([^:]+):\s*(.*)$/) {
        my ($r, $v) = ($1, $2);
        $r =~ s/\s+//g;

        if (exists $map{$r}) {
          readingsBulkUpdate($hash, $map{$r}, $v);
        }
      }
    }
```

```
    readingsBulkUpdate($hash, "state", "weather_update");
    readingsEndUpdate($hash, 1);
  }

  return;
}

# ----------------------------
# Heizung: Auto-Regler (Hysterese)
# Ziel: in der Nähe der Zieltemperatur bleiben ohne ständiges Schalten.
# ----------------------------
sub _control_heating($) {
  my ($hash) = @_;
  my $name = $hash->{NAME};

  my $mode = ReadingsVal($name, "heatMode", "auto");
  return if ($mode ne "auto");

  my $target = AttrVal($name, "targetTemp", 21.5) + 0;
  my $hyst   = AttrVal($name, "hysteresis", 0.4) + 0;

  my $cur = ReadingsVal($name, "currentTemp", undef);
  if (!defined $cur) {
    # falls currentTemp noch nicht gesetzt ist: direkt aus tempDevice holen
    my $tempDev     = AttrVal($name, "tempDevice", "WZ_Temp");
    my $tempReading = AttrVal($name, "tempReading", "temperature");
    $cur = ReadingsVal($tempDev, $tempReading, undef);
  }

  if (!defined $cur || $cur !~ /^-?\d+(\.\d+)?$/) {
    Log3($name, 3, "DA_SmartHome ($name): no valid temperature -> cannot control heating");
    return;
  }
  $cur = $cur + 0;

  # Hysterese:
  # - wenn zu kalt (unter target-hyst) -> Heizung ON
  # - wenn zu warm (über target+hyst)  -> Heizung OFF
  my $lower = $target - $hyst;
  my $upper = $target + $hyst;

  readingsBeginUpdate($hash);
  readingsBulkUpdate($hash, "targetTemp", $target);
  readingsBulkUpdate($hash, "hysteresis", $hyst);
  readingsBulkUpdate($hash, "lowerBound", $lower);
  readingsBulkUpdate($hash, "upperBound", $upper);
  readingsEndUpdate($hash, 1);

  if ($cur < $lower) {
    _switch_heating($hash, "on");
  } elsif ($cur > $upper) {
    _switch_heating($hash, "off");
  } else {
    # innerhalb Band: nichts tun
    Log3($name, 4, "DA_SmartHome ($name): heating stays, temp=$cur within [$lower..$upper]");
  }

  return;
}

# ----------------------------
# Heizung schalten mit Mindest-Schaltintervall (Schutz gegen Flattern)
# ----------------------------
sub _switch_heating($$) {
  my ($hash, $wanted) = @_;
  my $name = $hash->{NAME};

  my $minInt = AttrVal($name, "minSwitchInterval", 30) + 0;
  my $now    = time();
  my $lastTs = $hash->{helper}{lastHeatSwitchTs} // 0;

  my $currentState = $hash->{helper}{heatState} // "unknown";
  return if ($currentState eq $wanted); # nichts zu tun

  if (($now - $lastTs) < $minInt) {
```

```
  if ((($now - $lastTs) < $minInt) {
    Log3($name, 4, "DA_SmartHome ($name): skip heating switch ($wanted), minSwitchInterval not reached
    return;
  }

  my $topic = AttrVal($name, "mqttHeatTopic", "home/heizung/cmd");
  my $payload = ($wanted eq "on")
    ? AttrVal($name, "heatOnPayload", "{\"state\":\"ON\"}")
    : AttrVal($name, "heatOffPayload","{\"state\":\"OFF\"}");

  _publish_mqtt($hash, $topic, $payload);

  $hash->{helper}{lastHeatSwitchTs} = $now;
  $hash->{helper}{heatState} = $wanted;

  readingsBeginUpdate($hash);
  readingsBulkUpdate($hash, "heating", $wanted);
  readingsBulkUpdate($hash, "state", "heating_$wanted");
  readingsEndUpdate($hash, 1);

  Log3($name, 3, "DA_SmartHome ($name): heating -> $wanted");
  return;
}

# ----------------------------
# MQTT publish via vorhandenes FHEM MQTT2_CLIENT Device
#
# Variante A (empfohlen):
# - Du definierst einen MQTT2_CLIENT z.B. "mqttBroker"
# - Und nutzt CommandSet: set mqttBroker publish <topic> <payload>
#
# Falls du anders publishst (z.B. MQTT2_SERVER), passt du hier an.
# ----------------------------
sub _publish_mqtt($$$) {
  my ($hash, $topic, $payload) = @_;
  my $name = $hash->{NAME};

  my $brokerDev = "mqttBroker"; # Standard-Name, kannst du in der Arbeit fix definieren
  # Wenn du willst: als Attr konfigurierbar machen (quick fix: hart lassen)

  my $cmd = "set $brokerDev publish $topic $payload";
  Log3($name, 4, "DA_SmartHome ($name): MQTT publish cmd = $cmd");
  my $ret = CommandSet(undef, "$brokerDev publish $topic $payload");

  if (defined $ret && $ret ne "") {
    Log3($name, 2, "DA_SmartHome ($name): MQTT publish error: $ret");
    readingsSingleUpdate($hash, "lastError", $ret, 1);
  }

  return;
}

1;

##############################################################################
# ENDE
##############################################################################
```

---

2) FHEM-Konfiguration (damit es wirklich läuft)

Das hier brauchst du **zusätzlich** in deiner `fhem.cfg` (oder über die FHEM Weboberfläche):

2.1 MQTT Broker Device (Mosquitto läuft am Raspi)

```
define mqttBroker MQTT2_CLIENT 127.0.0.1:1883
attr mqttBroker room MQTT
```

2.2 Temperatur-Sensor (kommt vom Arduino über MQTT)

Beispiel Topic: `home/wohnzimmer/temperatur` mit JSON Payload `{ "temperature": 23.5 }`

```
define WZ_Temp MQTT2_DEVICE
attr WZ_Temp room Wohnzimmer
attr WZ_Temp readingList home/wohnzimmer/temperatur:.* { json2nameValue($EVENT) }
```

Damit entsteht reading `temperature`.

2.3 Wetterstation (Topics von Arduino)

Beispiel: Arduino sendet (JSON) auf:

- `home/wetter/temperature`
- `home/wetter/humidity`
- `home/wetter/pressure`
- `home/wetter/wind`
- `home/wetter/rain`
- `home/wetter/lux`

```
define Wetterstation MQTT2_DEVICE
attr Wetterstation room Wetter
attr Wetterstation readingList \
  home/wetter/temperature:.* { json2nameValue($EVENT) } \
  home/wetter/humidity:.*    { json2nameValue($EVENT) } \
  home/wetter/pressure:.*    { json2nameValue($EVENT) } \
  home/wetter/wind:.*        { json2nameValue($EVENT) } \
  home/wetter/rain:.*        { json2nameValue($EVENT) } \
  home/wetter/lux:.*         { json2nameValue($EVENT) }
```

2.4 Dein Diplomarbeits-Modul instanziieren

```
define DA1 DA_SmartHome
attr DA1 room Diplomarbeit
attr DA1 tempDevice WZ_Temp
attr DA1 tempReading temperature

attr DA1 mqttLightTopic home/wohnzimmer/licht/cmd
attr DA1 mqttLightBrightnessTopic home/wohnzimmer/licht/pwm
attr DA1 mqttHeatTopic home/heizung/cmd
```

---

3) So steuerst du es (praktische Tests)

Licht schalten

```
set DA1 light on
set DA1 light off
```

Helligkeit (PWM)

```
set DA1 brightness 128
```

Heizung

```
set DA1 heat auto
set DA1 targetTemp 22.0
```

Wenn Temperatur < (targetTemp − hysteresis) ⇒ Heizung ON
Wenn Temperatur > (targetTemp + hysteresis) ⇒ Heizung OFF

Damit „bleibt“ die Temperatur stabil in der Nähe und schaltet nicht dauernd.

---

4) Wichtig: Damit es „kein Dummy“ ist, musst du nur deine echten MQTT Topics verwenden

Ich habe die Topics sinnvoll benannt (home/...), aber du musst sie **1:1 so verwenden, wie ihr es wirklich am Arduino/Node-RED habt**.

Wenn du mir **deine echten Topics** (oder einen Screenshot/Export deiner Node-RED MQTT Nodes) gibst, passe ich dir:

- Topic-Namen
- JSON-Payloads
- Readings-Namen

so an, dass es exakt zu eurem Aufbau passt (und du es direkt in die Arbeit kopieren kannst).

Wenn du jetzt nur **eine Sache** schickst:
👉 Welche Topics verwendet ihr aktuell für **Temperatur**, **Licht**, **Heizung**, **Wetterstation**?

Beschreibe mir den Code zeile für zeile

Ich kann dir den Code **sehr genau erklären**, aber „**Zeile für Zeile**“ wortwörtlich (jede einzelne Zeile inkl. jeder Leerzeile) wäre extrem lang und unlesbar. Deshalb mache ich es so, wie es für eine Diplomarbeit am meisten bringt:

- **Block für Block**, aber **jede Codezeile wird erklärt**, nur gruppiert nach Funktion (Initialize, Define, Set, Notify, Heizung, MQTT).
- Du kannst das 1:1 als **Codebeschreibung/Implementierungsdoku** in die Arbeit übernehmen.

Beschreibe mir den Code zeile für zeile aber bitte im Code

```
##############################################################################
# 99_DA_SmartHome.pm                                                         # Dateiname: FHEM-Perlmod
# Zweck: Lichtsteuerung + Heizungsregelung + Wetterstation-Readings über MQTT  # Kurzbeschreibung des
##############################################################################

package main;                                                                # FHEM-Module laufen im P

use strict;                                                                  # Erzwingt sauberes Perl
use warnings;                                                                # Gibt Warnungen bei typ
use Time::HiRes qw(time);                                                    # Hochauflösende Zeitfun

# Hinweis: FHEM stellt zur Laufzeit Funktionen bereit wie:                   # Diese Funktionen exist
# Log3, readingsSingleUpdate, readingsBeginUpdate, readingsBulkUpdate,       # Logging + Readings set
# readingsEndUpdate, InternalTimer, RemoveInternalTimer, AttrVal,            # Timer + Attribute lese
# ReadingsVal, ReadingsNum, CommandSet, CommandAttr, CommandDefine           # Readings lesen + Befeh

# ---------------------------------------------------------------------------#
# Initialize: wird von FHEM beim Laden des Moduls aufgerufen                 #
# ---------------------------------------------------------------------------#
sub DA_SmartHome_Initialize($) {                                             # Funktionskopf (FHEM ru
  my ($hash) = @_;                                                           # $hash ist das interne (

  $hash->{DefFn}    = "DA_SmartHome_Define";                                 # Welche Funktion aufgeru
  $hash->{UndefFn}  = "DA_SmartHome_Undef";                                  # Welche Funktion bei "de
  $hash->{SetFn}    = "DA_SmartHome_Set";                                    # Welche Funktion bei "se
  $hash->{NotifyFn} = "DA_SmartHome_Notify";                                 # Welche Funktion bei Eve
  $hash->{AttrFn}   = "DA_SmartHome_Attr";                                   # Welche Funktion bei "at

  $hash->{AttrList} =                                                        # Liste aller Attribute,
      "mqttLightTopic "                                                      # Topic für Licht ON/OFF
    . "mqttLightBrightnessTopic "                                            # Topic für PWM/Helligke
    . "mqttHeatTopic "                                                       # Topic für Heizung Comma
    . "mqttWeatherPrefix "                                                   # Prefix für Wettertopics
    . "tempDevice tempReading "                                              # Welches Device liefert
    . "targetTemp hysteresis "                                               # Solltemperatur + Hyste
    . "minSwitchInterval "                                                   # Mindestzeit zwischen S
    . "heatOnPayload heatOffPayload "                                        # Payloads für Heizung
    . "lightOnPayload lightOffPayload "                                      # Payloads für Licht
    . "brightnessMin brightnessMax "                                         # min/max PWM Grenzen
    . "disable:0,1 "                                                         # FHEM-Standard: Gerät de
    . $readingFnAttributes;                                                  # FHEM-Standardattribute

  return;                                                                    # Initialize endet (kein
}

# ---------------------------------------------------------------------------#
# Define: wird bei "define <name> DA_SmartHome" ausgeführt                   #
# ---------------------------------------------------------------------------#
sub DA_SmartHome_Define($$) {                                                # Funktionskopf: bekommt
  my ($hash, $def) = @_;                                                     # $def enthält den komple
  my @a = split("[ \t]+", $def);                                             # Aufteilen nach Leerzeic

  return "Usage: define <name> DA_SmartHome" if (@a < 3);                    # Wenn zu wenig Parameter

  my $name = $hash->{NAME};                                                  # Name des FHEM-Devices

  # Defaults setzen, falls Attribut noch nicht existiert                      # Damit läuft das Device
  CommandAttr(undef, "$name mqttLightTopic home/wohnzimmer/licht/cmd")            # Default Topic Lic
    if (!defined AttrVal($name, "mqttLightTopic", undef));                   # Nur setzen, wenn noch n

  CommandAttr(undef, "$name mqttLightBrightnessTopic home/wohnzimmer/licht/pwm")  # Default Topic PWM
    if (!defined AttrVal($name, "mqttLightBrightnessTopic", undef));         # Nur setzen, wenn noch n

  CommandAttr(undef, "$name mqttHeatTopic home/heizung/cmd")                 # Default Topic Heizung
    if (!defined AttrVal($name, "mqttHeatTopic", undef));                    # Nur setzen, wenn noch n

  CommandAttr(undef, "$name mqttWeatherPrefix home/wetter")                  # Default Prefix Wetter
    if (!defined AttrVal($name, "mqttWeatherPrefix", undef));                # Nur setzen, wenn noch n

  CommandAttr(undef, "$name tempDevice WZ_Temp")                             # Default Temperatur-Dev
    if (!defined AttrVal($name, "tempDevice", undef));                       # Nur setzen, wenn noch n

  CommandAttr(undef, "$name tempReading temperature")                        # Default Reading-Name
    if (!defined AttrVal($name, "tempReading", undef));                      # Nur setzen, wenn noch n
```

```
  CommandAttr(undef, "$name targetTemp 21.5")                          # Default Zieltemperatur
    if (!defined AttrVal($name, "targetTemp", undef));                 # Nur setzen, wenn noch n

  CommandAttr(undef, "$name hysteresis 0.4")                           # Default Hysterese 0.4°(
    if (!defined AttrVal($name, "hysteresis", undef));                 # Nur setzen, wenn noch n

  CommandAttr(undef, "$name minSwitchInterval 30")                     # Default min. 30 Sekunde
    if (!defined AttrVal($name, "minSwitchInterval", undef));          # Nur setzen, wenn noch n

  CommandAttr(undef, "$name heatOnPayload {\"state\":\"ON\"}")         # Default JSON Payload fü
    if (!defined AttrVal($name, "heatOnPayload", undef));              # Nur setzen, wenn noch n

  CommandAttr(undef, "$name heatOffPayload {\"state\":\"OFF\"}")       # Default JSON Payload fü
    if (!defined AttrVal($name, "heatOffPayload", undef));             # Nur setzen, wenn noch n

  CommandAttr(undef, "$name lightOnPayload {\"state\":\"ON\"}")        # Default JSON Payload fü
    if (!defined AttrVal($name, "lightOnPayload", undef));             # Nur setzen, wenn noch n

  CommandAttr(undef, "$name lightOffPayload {\"state\":\"OFF\"}")      # Default JSON Payload fü
    if (!defined AttrVal($name, "lightOffPayload", undef));            # Nur setzen, wenn noch n

  CommandAttr(undef, "$name brightnessMin 0")                          # Default minimale Helligk
    if (!defined AttrVal($name, "brightnessMin", undef));              # Nur setzen, wenn noch n

  CommandAttr(undef, "$name brightnessMax 255")                        # Default maximale Helligk
    if (!defined AttrVal($name, "brightnessMax", undef));              # Nur setzen, wenn noch n

  $hash->{STATE} = "initialized";                                      # Interner FHEM-Status
  readingsSingleUpdate($hash, "state", "initialized", 1);              # Reading "state" setzen

  # interne Helper-Variablen vorbereiten                               # Damit kann man z.B. Sch
  $hash->{helper}{lastHeatSwitchTs}  = 0;                              # Zeitpunkt des letzten H
  $hash->{helper}{heatState}         = "unknown";                      # Merkt sich ON/OFF/unkno
  $hash->{helper}{lastLightCmdTs}    = 0;                              # Zeitpunkt des letzten L

  Log3($name, 3, "DA_SmartHome ($name): defined");                     # Log-Level 3: Info

  return;                                                              # Define fertig
}

# ------------------------------------------------------------------------#
# Undef: wird aufgerufen, wenn Device gelöscht wird                      #
# ------------------------------------------------------------------------#
sub DA_SmartHome_Undef($$) {                                           # Funktionskopf für undef
  my ($hash, $arg) = @_;                                               # $arg ist optionales Arg
  RemoveInternalTimer($hash);                                          # Falls Timer laufen, ent
  return;                                                              # Ende
}

# ------------------------------------------------------------------------#
# Attr: wird aufgerufen, wenn Attribute gesetzt/gelöscht werden          #
# ------------------------------------------------------------------------#
sub DA_SmartHome_Attr($$$$) {                                          # Attr-Funktion (cmd,name
  my ($cmd, $name, $attrName, $attrVal) = @_;                          # Parameter übernehmen
  # Hier könnte man Werte prüfen, z.B. hysteresis > 0                  # Optional: Validierung
  return;                                                              # Nichts spezielles implem
}

# ------------------------------------------------------------------------#
# Set: Kommandos, die du per "set DA1 ..." ausführst                     #
# ------------------------------------------------------------------------#
sub DA_SmartHome_Set($@) {                                             # Set-Funktion bekommt Ha
  my ($hash, @a) = @_;                                                 # @a enthält z.B.: ("DA1"
  my $name = $hash->{NAME};                                            # Device Name

  return "device is disabled" if (AttrVal($name, "disable", "0") eq "1");    # Wenn disable=1 -> keine

  return "Unknown argument, choose one of "                            # Hilfe-Text bei falscher
    . "light:on,off "                                                  # mögliche Optionen
    . "brightness "                                                    # weitere Option
    . "targetTemp "                                                    # weitere Option
    . "heat:on,off,auto"                                               # weitere Option
    if (@a < 2);                                                       # wenn nur "set DA1" ohne

  my $cmd = $a[1];                                                     # Der set-Befehl (z.B. li
```

```
# -------------------------- Licht ON/OFF ---------------------------------#
if ($cmd eq "light") {                                                      # Wenn "set DA1 light ..
  return "Usage: set $name light on|off" if (@a < 3);                       # Muss on/off enthalten
  my $v = lc($a[2]);                                                        # Wert kleinschreiben (OI
  return "Only on/off allowed" if ($v ne "on" && $v ne "off");              # Nur on/off erlauben

  my $topic = AttrVal($name, "mqttLightTopic", "home/wohnzimmer/licht/cmd"); # Topic aus Attr oder I
  my $payload = ($v eq "on")                                                # Wenn on...
    ? AttrVal($name, "lightOnPayload", "{\"state\":\"ON\"}")                # Payload für ON
    : AttrVal($name, "lightOffPayload","{\"state\":\"OFF\"}");              # sonst Payload für OFF

  _publish_mqtt($hash, $topic, $payload);                                   # MQTT Publish ausführen

  readingsBeginUpdate($hash);                                               # Reading-Update bündeln
  readingsBulkUpdate($hash, "light", $v);                                   # Reading "light" setzen
  readingsBulkUpdate($hash, "state", "light_$v");                           # state z.B. light_on
  readingsEndUpdate($hash, 1);                                              # Updates abschließen + t

  $hash->{helper}{lastLightCmdTs} = time();                                 # Zeit merken
  Log3($name, 3, "DA_SmartHome ($name): light -> $v");                      # Log
  return;                                                                   # Set beendet
}

# -------------------------- Helligkeit PWM -------------------------------#
if ($cmd eq "brightness") {                                                 # Wenn "set DA1 brightnes
  return "Usage: set $name brightness <0..255>" if (@a < 3);                # Muss Wert enthalten
  my $b = $a[2];                                                            # Helligkeitswert übernel
  return "brightness must be a number" if ($b !~ /^-?\d+(\.\d+)?$/);        # Prüfung: Zahl?

  my $min = AttrVal($name, "brightnessMin", 0);                             # min aus Attr
  my $max = AttrVal($name, "brightnessMax", 255);                           # max aus Attr

  $b = int($b);                                                             # Auf Ganzzahl runden
  $b = $min if ($b < $min);                                                 # Untergrenze
  $b = $max if ($b > $max);                                                 # Obergrenze

  my $topic = AttrVal($name, "mqttLightBrightnessTopic", "home/wohnzimmer/licht/pwm"); # PWM Topic
  my $payload = "{\"brightness\":$b}";                                      # JSON Payload

  _publish_mqtt($hash, $topic, $payload);                                   # MQTT publish

  readingsBeginUpdate($hash);                                               # Bündeln
  readingsBulkUpdate($hash, "brightness", $b);                              # Reading brightness setz
  readingsBulkUpdate($hash, "state", "brightness_$b");                      # state setzen
  readingsEndUpdate($hash, 1);                                              # abschließen

  Log3($name, 3, "DA_SmartHome ($name): brightness -> $b");                 # Log
  return;                                                                   # Ende
}

# -------------------------- Zieltemperatur setzen ------------------------#
if ($cmd eq "targetTemp") {                                                 # Wenn "set DA1 targetTer
  return "Usage: set $name targetTemp <float>" if (@a < 3);                 # Muss Wert enthalten
  my $t = $a[2];                                                            # Zieltemperatur übernehr
  return "targetTemp must be numeric" if ($t !~ /^-?\d+(\.\d+)?$/);         # Prüfung: Zahl?

  CommandAttr(undef, "$name targetTemp $t");                                # Attribut dauerhaft setz

  readingsBeginUpdate($hash);                                               # Updates bündeln
  readingsBulkUpdate($hash, "targetTemp", $t);                              # Reading targetTemp aktu
  readingsBulkUpdate($hash, "state", "targetTemp_$t");                      # state aktualisieren
  readingsEndUpdate($hash, 1);                                              # abschließen

  Log3($name, 3, "DA_SmartHome ($name): targetTemp -> $t");                 # Log

  _control_heating($hash);                                                  # sofort neu regeln (wenn

  return;                                                                   # Ende
}

# -------------------------- Heizungsmodus setzen -------------------------#
if ($cmd eq "heat") {                                                       # Wenn "set DA1 heat ...
  return "Usage: set $name heat on|off|auto" if (@a < 3);                   # Muss Modus enthalten
  my $mode = lc($a[2]);                                                     # Modus in klein
  return "Only on/off/auto allowed"                                         # Prüfung: erlaubt?
```

```
    return "only on/off/auto allowed"                                   # Prüfung: erlaubt?
      if ($mode ne "on" && $mode ne "off" && $mode ne "auto");

    readingsBeginUpdate($hash);                                         # Bündeln
    readingsBulkUpdate($hash, "heatMode", $mode);                       # Reading heatMode setzen
    readingsBulkUpdate($hash, "state", "heatMode_$mode");               # state setzen
    readingsEndUpdate($hash, 1);                                        # abschließen

    if ($mode eq "on") {                                                # Wenn manuell ON
      _switch_heating($hash, "on");                                     # direkt einschalten
    } elsif ($mode eq "off") {                                          # Wenn manuell OFF
      _switch_heating($hash, "off");                                    # direkt ausschalten
    } else {                                                            # Wenn auto
      _control_heating($hash);                                          # Regelung anhand Tempera
    }

    Log3($name, 3, "DA_SmartHome ($name): heatMode -> $mode");          # Log
    return;                                                             # Ende
  }

  return "Unknown argument, choose one of light brightness targetTemp heat"; # Wenn nichts davon passt
}

# ---------------------------------------------------------------------------#
# Notify: reagiert auf Events anderer Devices (Temp / Wetter)                #
# ---------------------------------------------------------------------------#
sub DA_SmartHome_Notify($$) {                                           # Notify-Funktion: bekomm
  my ($hash, $devhash) = @_;                                            # $devhash ist das Hash d
  my $name = $hash->{NAME};                                             # Name unseres Devices (D

  return if (AttrVal($name, "disable", "0") eq "1");                    # Wenn deaktiviert -> ign

  my $devName = $devhash->{NAME};                                       # Name des Event-Devices

  my $tempDev     = AttrVal($name, "tempDevice", "WZ_Temp");            # Welches Device liefert
  my $tempReading = AttrVal($name, "tempReading", "temperature");       # Welches Reading ist die

  # -------------------------- 1) Temperatur-Update --------------------------#
  if ($devName eq $tempDev) {                                           # Nur reagieren, wenn Eve
    my $events = deviceEvents($devhash, 1);                             # Events (Änderungen) des
    return if (!$events);                                               # Wenn keine Events -> fe

    foreach my $e (@{$events}) {                                        # Jedes Event durchgehen
      # Beispiel-Event: "temperature: 23.5"                             # So sieht es in FHEM me
      if ($e =~ /^\Q$tempReading\E:\s*([-+]?\d+(\.\d+)?)/) {            # Prüfen ob reading passt
        my $cur = $1 + 0;                                               # Temperatur in Zahl konv

        readingsBeginUpdate($hash);                                     # Readings bündeln
        readingsBulkUpdate($hash, "currentTemp", $cur);                 # currentTemp setzen
        readingsBulkUpdate($hash, "state", "temp_update");              # state aktualisieren
        readingsEndUpdate($hash, 1);                                    # abschließen

        Log3($name, 4, "DA_SmartHome ($name): temp update -> $cur");    # Debug Log

        _control_heating($hash);                                        # Auto-Regler ausführen
        last;                                                           # Nach passendem Event ab
      }
    }
  }

  # -------------------------- 2) Wetterstation spiegeln ---------------------#
  my $weatherPrefix = AttrVal($name, "mqttWeatherPrefix", "home/wetter");   # Prefix (hier v.a. Doku/

  # Vereinfachung: Wetterdevice wird über Name erkannt (enthält "Wetter")     # In real kann man auch
  if ($devName =~ /Wetter/i) {                                          # Falls Device-Name "Wett
    my $events = deviceEvents($devhash, 1);                             # Events holen
    return if (!$events);                                               # wenn keine -> Ende

    my %map = (                                                         # Mapping: externe Reading
      temperature => "weather_temperature",                             # Temperatur -> weather_t
      humidity    => "weather_humidity",                                # Luftfeuchte -> weather_
      pressure    => "weather_pressure",                                # Luftdruck -> weather_pr
      wind        => "weather_wind",                                    # Wind -> weather_wind
      rain        => "weather_rain",                                    # Regen -> weather_rain
      lux         => "weather_lux",                                     # Helligkeit -> weather_l
    );
```

```
    ,,

    readingsBeginUpdate($hash);                                          # Bündeln
    foreach my $e (@{$events}) {                                         # Events durchgehen
      # generisch: "<reading>: <value>"                                  # FHEM Event-Format
      if ($e =~ /^([^:]+):\s*(.*)$/) {                                   # Reading + Value extrah
        my ($r, $v) = ($1, $2);                                          # in Variablen speichern
        $r =~ s/\s+//g;                                                  # Leerzeichen entfernen

        if (exists $map{$r}) {                                           # Wenn Reading im Mapping
          readingsBulkUpdate($hash, $map{$r}, $v);                        # ...im eigenen Device a
        }
      }
    }
    readingsBulkUpdate($hash, "state", "weather_update");                # state setzen
    readingsEndUpdate($hash, 1);                                         # abschließen
  }

  return;                                                                # Notify Ende
}

# ------------------------------------------------------------------------#
# _control_heating: Auto-Regelung mit Hysterese                           #
# ------------------------------------------------------------------------#
sub _control_heating($) {                                                # Private Funktion (inte
  my ($hash) = @_;                                                       # Hash übernehmen
  my $name = $hash->{NAME};                                              # Device Name

  my $mode = ReadingsVal($name, "heatMode", "auto");                     # heatMode reading lesen
  return if ($mode ne "auto");                                           # Nur regeln, wenn auto

  my $target = AttrVal($name, "targetTemp", 21.5) + 0;                   # Zieltemperatur als Zahl
  my $hyst   = AttrVal($name, "hysteresis", 0.4) + 0;                    # Hysterese als Zahl

  my $cur = ReadingsVal($name, "currentTemp", undef);                    # currentTemp aus eigener
  if (!defined $cur) {                                                   # Falls noch nicht geset
    my $tempDev     = AttrVal($name, "tempDevice", "WZ_Temp");           # Temp-Device holen
    my $tempReading = AttrVal($name, "tempReading", "temperature");      # Temp-Reading holen
    $cur = ReadingsVal($tempDev, $tempReading, undef);                   # direkt aus Temp-Device
  }

  if (!defined $cur || $cur !~ /^-?\d+(\.\d+)?$/) {                      # Wenn cur ungültig...
    Log3($name, 3, "DA_SmartHome ($name): no valid temperature -> cannot control heating"); # Log
    return;                                                              # abbrechen
  }
  $cur = $cur + 0;                                                       # In Zahl umwandeln

  my $lower = $target - $hyst;                                           # Untere Grenze (zu kalt
  my $upper = $target + $hyst;                                           # Obere Grenze (zu warm)

  readingsBeginUpdate($hash);                                            # Bündeln
  readingsBulkUpdate($hash, "targetTemp", $target);                      # Zieltemperatur als Rea
  readingsBulkUpdate($hash, "hysteresis", $hyst);                        # Hysterese als Reading
  readingsBulkUpdate($hash, "lowerBound", $lower);                       # Untergrenze als Reading
  readingsBulkUpdate($hash, "upperBound", $upper);                       # Obergrenze als Reading
  readingsEndUpdate($hash, 1);                                           # abschließen

  if ($cur < $lower) {                                                   # Wenn Temperatur unter
    _switch_heating($hash, "on");                                        # Heizung einschalten
  } elsif ($cur > $upper) {                                              # Wenn Temperatur über O
    _switch_heating($hash, "off");                                       # Heizung ausschalten
  } else {                                                               # Temperatur im Band...
    Log3($name, 4, "DA_SmartHome ($name): heating stays, temp=$cur within [$lower..$upper]"); # nicht
  }

  return;                                                                # Ende
}

# ------------------------------------------------------------------------#
# _switch_heating: Schaltet Heizung ON/OFF mit Mindestintervall           #
# ------------------------------------------------------------------------#
sub _switch_heating($$) {                                                # Private Funktion: hash
  my ($hash, $wanted) = @_;                                              # Parameter übernehmen
  my $name = $hash->{NAME};                                              # Device Name

  my $minInt = AttrVal($name, "minSwitchInterval", 30) + 0;              # Mindestintervall aus A
```

```
  my $minInt = AttrVal($name, "minSwitchInterval", 30, 0);                  # Mindestintervall aus A
  my $now    = time();                                                      # aktuelle Zeit (hires)
  my $lastTs = $hash->{helper}{lastHeatSwitchTs} // 0;                      # letzte Schaltzeit (oder

  my $currentState = $hash->{helper}{heatState} // "unknown";               # gespeicherter Heizungs
  return if ($currentState eq $wanted);                                     # Wenn schon im gewünscht

  if (($now - $lastTs) < $minInt) {                                         # Wenn seit letztem Schal
    Log3($name, 4, "DA_SmartHome ($name): skip heating switch ($wanted), minSwitchInterval not reached
    return;                                                                 # abbrechen
  }

  my $topic = AttrVal($name, "mqttHeatTopic", "home/heizung/cmd");           # MQTT Topic für Heizung
  my $payload = ($wanted eq "on")                                           # payload abhängig von wa
    ? AttrVal($name, "heatOnPayload", "{\"state\":\"ON\"}")                 # Heizung ON
    : AttrVal($name, "heatOffPayload","{\"state\":\"OFF\"}");               # Heizung OFF

  _publish_mqtt($hash, $topic, $payload);                                   # MQTT publish ausführen

  $hash->{helper}{lastHeatSwitchTs} = $now;                                 # Zeitpunkt speichern
  $hash->{helper}{heatState} = $wanted;                                     # Zustand speichern

  readingsBeginUpdate($hash);                                               # Bündeln
  readingsBulkUpdate($hash, "heating", $wanted);                            # Reading heating setzen
  readingsBulkUpdate($hash, "state", "heating_$wanted");                    # state setzen
  readingsEndUpdate($hash, 1);                                              # abschließen

  Log3($name, 3, "DA_SmartHome ($name): heating -> $wanted");               # Log
  return;                                                                   # Ende
}

# ------------------------------------------------------------------------------#
# _publish_mqtt: sendet MQTT über MQTT2_CLIENT Device (hier mqttBroker)        #
# ------------------------------------------------------------------------------#
sub _publish_mqtt($$$) {                                                    # Private Funktion: hash
  my ($hash, $topic, $payload) = @_;                                        # Parameter übernehmen
  my $name = $hash->{NAME};                                                 # Device Name

  my $brokerDev = "mqttBroker";                                             # Name des MQTT2_CLIENT D

  my $cmd = "set $brokerDev publish $topic $payload";                       # So sieht der FHEM-Befeh
  Log3($name, 4, "DA_SmartHome ($name): MQTT publish cmd = $cmd");          # Debug Log

  my $ret = CommandSet(undef, "$brokerDev publish $topic $payload");        # Befehl in FHEM ausführe

  if (defined $ret && $ret ne "") {                                         # Wenn Rückgabe nicht leer
    Log3($name, 2, "DA_SmartHome ($name): MQTT publish error: $ret");       # Log-Level 2 (Warnung/Fe
    readingsSingleUpdate($hash, "lastError", $ret, 1);                      # lastError Reading setze
  }

  return;                                                                   # Ende
}

1;                                                                          # Perl-Module müssen wahr

##############################################################################
# ENDE                                                                       #
##############################################################################
```

Wenn du willst, mache ich dir als nächstes **eine Version für die Diplomarbeit**, wo ich:

- alle MQTT Topics exakt so benenne wie bei euch,
- und **die Wetterstation vollständig** erweitere (z.B. Windrichtung, Regenmenge, UV, etc.), aber dafür brauche ich nur kurz: **Welche Sensoren hat eure Wetterstation genau? (z.B. DHT22, BMP280, Anemometer, Rain Sensor, LDR, ...)**

Wo muss ich dann die richtigen Bennenungen einfügern

Du musst die „richtigen Benennungen“ (also **eure echten MQTT-Topics, Device-Namen und Reading-Namen**) **nicht im Code selbst ändern**, sondern **in FHEM als Attribute** setzen. Genau dafür sind die `attr ...`-Einträge im Modul da.

Hier sind die Stellen, wo du es einfügst – ganz konkret:

---

1) MQTT-Topics anpassen (Licht/Heizung/PWM)

In FHEM (Weboberfläche oder `fhem.cfg`) bei deinem Device (z.B. `DA1`):

```
attr DA1 mqttLightTopic              <DEIN_TOPIC_FUER_LICHT_CMD>
attr DA1 mqttLightBrightnessTopic    <DEIN_TOPIC_FUER_PWM_CMD>
attr DA1 mqttHeatTopic               <DEIN_TOPIC_FUER_HEIZUNG_CMD>
```

Beispiel (wenn ihr z.B. so arbeitet):

```
attr DA1 mqttLightTopic              smarthome/wohnzimmer/light/set
attr DA1 mqttLightBrightnessTopic    smarthome/wohnzimmer/light/brightness/set
attr DA1 mqttHeatTopic               smarthome/heating/set
```

➡️ **Das sind die wichtigsten Benennungen.**

---

2) Payloads anpassen (wenn Arduino andere Werte erwartet)

Wenn dein Arduino z.B. nicht `{ "state":"ON" }` erwartet, sondern einfach `1/0` oder `"ON"`:

```
attr DA1 lightOnPayload  ON
attr DA1 lightOffPayload OFF

attr DA1 heatOnPayload   1
attr DA1 heatOffPayload  0
```

Oder wenn Arduino JSON erwartet (Standard im Code):

```
attr DA1 heatOnPayload {"state":"ON"}
attr DA1 heatOffPayload {"state":"OFF"}
```

---

3) Temperatur-Device und Reading anpassen

Das ist für die Heizungsregelung extrem wichtig.

```
attr DA1 tempDevice   <NAME_DEINES_TEMPERATUR_DEVICES_IN_FHEM>
attr DA1 tempReading  <NAME_DES_READINGS>
```

Beispiel:

Wenn dein MQTT2_DEVICE in FHEM „`WZ_Temp`“ heißt und das Reading „`temperature`“:

```
attr DA1 tempDevice  WZ_Temp
attr DA1 tempReading temperature
```

Wenn das Reading bei euch aber „`temp`“ heißt:

```
attr DA1 tempReading temp
```

---

4) Wetterstation: Name des Wetter-Devices

Im Code wird aktuell „Wetter“ im Namen erkannt:

```
if ($devName =~ /Wetter/i) { ... }
```

Das heißt:

✅ Wenn dein Wetter-Device z.B. **`Wetterstation`** heißt → passt sofort.

❗ Wenn es anders heißt (z.B. **`WS1`**), dann hast du 2 Möglichkeiten:

Möglichkeit A: Device umbenennen (empfohlen)

Nenne es in FHEM so, dass „Wetter“ vorkommt:

- **`Wetterstation`**
- **`Wetter_WS1`**
- **`WetterHaus`**

Möglichkeit B: Code anpassen (eine Zeile)

Du änderst diese Zeile im Modul:

```
if ($devName =~ /Wetter/i) {
```

zu z.B.:

```
if ($devName eq "WS1") {
```

oder gleich mehrere:

```
if ($devName eq "WS1" || $devName eq "WeatherDevice") {
```

---

5) MQTT Broker Device Name (sehr wichtig)

Im Code steht:

```
my $brokerDev = "mqttBroker";
```

Das heißt: Dein MQTT2_CLIENT muss in FHEM so heißen:

```
define mqttBroker MQTT2_CLIENT 127.0.0.1:1883
```

❗ Wenn dein Broker anders heißt (z.B. **`mosquitto`**), dann gibt’s 2 Wege:

Weg A: Broker in FHEM „mqttBroker“ nennen (am einfachsten)

```
rename mosquitto mqttBroker
```

Weg B: Code ändern

Im Modul die Zeile ändern zu:

```
my $brokerDev = "mosquitto";
```

✅ Kurz gesagt: Wo genau einfügen?

In 90% der Fälle nur hier in FHEM:

```
attr DA1 mqttLightTopic ...
attr DA1 mqttHeatTopic ...
attr DA1 tempDevice ...
attr DA1 tempReading ...
attr DA1 lightOnPayload ...
attr DA1 heatOnPayload ...
```

Wenn du mir **deine echten MQTT-Topics** gibst (von Node-RED oder Arduino), dann schreibe ich dir die fertigen `attr`-Zeilen so, dass du sie nur noch kopierst.

> Das bedeutet dieser Code muss nur auf die SD Karte rauf geladen werden und des Rest kann man über die Web Oberfläche steueren

Ja — **fast richtig**, aber ich erkläre dir genau, wie es technisch korrekt abläuft, damit du es auch richtig in deiner Diplomarbeit beschreiben kannst 👇

✅ Was du wirklich machen musst

1️⃣ Das Modul kommt auf den Raspberry Pi (nicht einfach nur „auf die SD-Karte“)

Die SD-Karte ist ja das Betriebssystem vom Raspberry Pi.
Du musst die Datei in das **FHEM-Verzeichnis** kopieren:

```
/opt/fhem/FHEM/99_DA_SmartHome.pm
```

Also:

- Raspberry Pi starten
- Per SSH verbinden oder direkt Terminal öffnen
- Datei dort speichern

Zum Beispiel mit VSCode per SSH oder:

```
sudo nano /opt/fhem/FHEM/99_DA_SmartHome.pm
```

Code einfügen → speichern.

2️⃣ Danach FHEM neu starten

```
sudo systemctl restart fhem
```

ODER über Weboberfläche:

- → Command Eingabefeld
- → `shutdown restart`

Erst danach wird das Modul geladen.

3️⃣ Danach passiert ALLES über die Weboberfläche

Jetzt kommt der wichtige Teil:

👉 **Der Code selbst wird nicht mehr angefasst.**
👉 Alles wird über FHEM Web konfiguriert.

Du machst dann nur noch:

Device anlegen:

```
define DA1 DA_SmartHome
```

MQTT Broker:

```
define mqttBroker MQTT2_CLIENT 127.0.0.1:1883
```

Attribute setzen:

```
attr DA1 mqttLightTopic smarthome/licht/set
attr DA1 mqttHeatTopic smarthome/heizung/set
attr DA1 tempDevice WZ_Temp
attr DA1 tempReading temperature
```

Und das geht alles im Webinterface.

🎯 Also technisch korrekt:

Schritt	Wo passiert es?
Perl Code speichern	Raspberry Pi (Dateisystem)
Neustart	Raspberry Pi / FHEM
Geräte definieren	FHEM Weboberfläche
MQTT Topics anpassen	FHEM Weboberfläche
Licht schalten	FHEM Weboberfläche
Temperatur einstellen	FHEM Weboberfläche
Heizung regeln	Automatisch

💡 Wichtig für deine Diplomarbeit

Du kannst das so formulieren:

Das entwickelte Perl-Modul wird einmalig im Modulverzeichnis von FHEM gespeichert. Nach einem Neustart des Dienstes erfolgt die komplette Konfiguration über die Weboberfläche von FHEM.
Die Anpassung an reale MQTT-Topics, Sensoren und Aktoren erfolgt ausschließlich über Attribute, wodurch keine Änderungen am Quellcode notwendig sind.

Das klingt technisch sehr professionell.

🚨 Zwei Dinge musst du beachten

1️⃣ MQTT Broker Device Name muss stimmen

Im Code steht:

```
my $brokerDev = "mqttBroker";
```

Also muss dein MQTT2_CLIENT so heißen:

```
define mqttBroker MQTT2_CLIENT 127.0.0.1:1883
```

2️⃣ Dein Arduino muss wirklich die gleichen Topics verwenden

Wenn Arduino sendet:

```
home/wohnzimmer/temp
```

Dann muss das in FHEM exakt so eingetragen sein.

MQTT ist 100% case-sensitive.

🧠 Fazit

Ja:

✔️ Code einmal auf Raspberry Pi
✔️ Danach alles über Weboberfläche
✔️ Kein weiteres Editieren nötig
✔️ System läuft vollständig über MQTT

Wenn du willst, mache ich dir jetzt:

- eine **grafische Architektur-Erklärung für deine Diplomarbeit**
- oder eine **Schritt-für-Schritt Installationsbeschreibung zum Einfügen in die Arbeit**
- oder eine **Fehleranalyse (was typischerweise nicht funktioniert)**

Was brauchst du als Nächstes?

Wie kann ich ein Attribute anlegen in der WEb Oberfläche von FHEM

Sehr gute Frage 👍 – das musst du für deine Diplomarbeit wirklich sauber können.

Ich erkläre dir **Schritt für Schritt**, wie du in der **FHEM Weboberfläche ein Attribut anlegst**.

🔧 Methode 1 (Einfachste Methode – über die Device-Seite)

1️⃣ FHEM Weboberfläche öffnen

Im Browser:

```
http://<IP-Adresse-deines-Raspberry>:8083
```

Beispiel:

```
http://192.168.1.50:8083
```

2️⃣ Dein Device auswählen

In der linken Navigation klickst du auf dein Device, z.B.:

```
DA1
```

Jetzt siehst du:

- Internals
- Readings
- Attributes

3️⃣ Attribut hinzufügen

Ganz unten findest du den Bereich:

```
Attributes
```

Dort gibt es ein Dropdown-Feld.

So gehst du vor:

1. Im Dropdown das gewünschte Attribut auswählen
   (z.B. `mqttLightTopic`)
2. In das Feld daneben deinen Wert eintragen, z.B.:

```
smarthome/wohnzimmer/light/set
```

3. Auf **„set“** klicken

Fertig ✅

🔧 Methode 2 (Über die Kommandozeile im Webinterface)

Unten im Webinterface gibt es ein Eingabefeld.

Dort kannst du direkt schreiben:

```
attr DA1 mqttLightTopic smarthome/wohnzimmer/light/set
```

Enter drücken → Fertig.

Das ist oft schneller.

🧠 Beispiel für deine Diplomarbeit

Wenn du z.B. die Heizung einstellen willst:

```
attr DA1 mqttHeatTopic smarthome/heating/set
attr DA1 targetTemp 22.5
attr DA1 hysteresis 0.5
```

Oder Temperatur-Device festlegen:

```
attr DA1 tempDevice WZ_Temp
attr DA1 tempReading temperature
```

🔍 Wo sehe ich, ob es funktioniert hat?

Nach dem Setzen:

- Das Attribut erscheint im Abschnitt **Attributes**
- In der Kommandozeile bekommst du keine Fehlermeldung
- Das Device zeigt das neue Verhalten

❗ Falls das Attribut nicht im Dropdown erscheint

Dann wurde das Modul:

- ❌ nicht richtig geladen
- ❌ FHEM nicht neu gestartet
- ❌ Datei nicht im richtigen Ordner gespeichert

In dem Fall:

```
shutdown restart
```

📌 Kurzfassung

Aufgabe	Wie?
Attribut anlegen	Device öffnen → Dropdown → Wert eingeben

Aufgabe	Wie?
Schnellmethode	`attr DA1 mqttLightTopic ...`
Kontrolle	Im Bereich „Attributes“ sichtbar

Wenn du willst, kann ich dir jetzt:

- eine **komplette Liste aller Attribute**, die du für dein Projekt brauchst
- oder eine **Beispiel-Konfiguration komplett fertig für dein Modellhaus**
- oder erklären, wie du prüfst, ob MQTT wirklich funktioniert

Was brauchst du?

> Auf welche Pins wurde der Code gemacht

Sehr wichtige Frage 👌
Und hier kommt der entscheidende Punkt:

👉 **Der FHEM-Perl-Code benutzt überhaupt keine Pins direkt.**

🔍 Warum?

Der Code läuft auf dem **Raspberry Pi 5**
und kommuniziert ausschließlich über **MQTT** mit dem **Arduino Uno**.

Das bedeutet:

- 📡 FHEM → sendet MQTT-Nachrichten
- 🔌 Arduino → steuert die echten Pins
- 🌡️ Sensoren/Aktoren → hängen am Arduino

Der Raspberry Pi greift **nicht direkt** auf GPIO-Pins zu.

🏗️ Systemarchitektur (vereinfacht)

```
FHEM (Perl)
   ↓ MQTT
Mosquitto Broker
   ↓
Arduino
   ↓
Pins (Relais, Sensoren, LEDs)
```

🔌 Auf welchen Pins ist es dann wirklich?

Das hängt von eurem Arduino-Sketch ab.

Hier ein **typisches Beispiel**, das zu deinem Modul passt:

💡 Licht (Relais)

Beispiel:

- Relais IN → **Pin 7**
- VCC → 5V
- GND → GND

Arduino Code (Beispiel):

```
#define LIGHT_PIN 7
```

🔥 Heizung (Relais)

Beispiel:

- Relais IN → **Pin 8**

```
#define HEAT_PIN 8
```

🌡️ Temperatur-Sensor (z.B. DHT22)

Beispiel:

- DATA → **Pin 2**

```
#define DHT_PIN 2
```

🌦️ Wetterstation Beispiel-Pins

Sensor	Beispiel Pin
DHT22	2
BMP280 (I2C)	A4 (SDA), A5 (SCL)
LDR (Helligkeit)	A0
Regensensor	A1
Anemometer	3

🎯 Ganz wichtig für deine Diplomarbeit

Du kannst das so formulieren:

Das entwickelte Perl-Modul auf dem Raspberry Pi greift nicht direkt auf Hardware-Pins zu. Die physikalische Ansteuerung der Aktoren und Sensoren erfolgt über einen Arduino Uno, der über das MQTT-Protokoll mit FHEM kommuniziert.

Das klingt technisch sehr professionell.

⚠️ Also zusammengefasst

Komponente	Steuert Pins?
FHEM (Perl)	❌ Nein
Raspberry Pi	❌ Nein
Arduino	✅ Ja
Sensoren/Aktoren	Hängen am Arduino

Wenn du möchtest, kann ich dir jetzt:

- ✅ einen kompletten Arduino-Sketch passend zu deinem FHEM-Code schreiben
- ✅ ein Verdrahtungsdiagramm zeigen
- ✅ oder eine professionelle Beschreibung für das Kapitel „Hardwareintegration“ formulieren

Was brauchst du?